

Python 응용: API 활용 국내 여행지 추천 프로그램 개발 최종 제출 보고서

— LLM API(Gemini) × 지도/장소 검색 API(Kakao Local) 연동 —

1. 미션 개요

현대 서비스 대부분은 API를 통해 데이터를 주고받는다. 본 미션은 단일 API 호출을 넘어, LLM API와 지도/장소 검색 API를 조합하여 하나의 인사이트(여행 리포트)를 만들어내는 흐름을 직접 구현하는 것을 목표로 한다.

사용자가 여행 날짜를 입력하면

- ① Gemini API가 해당 시기에 여행하기 좋은 국내 지역을 추천하고,
- ② Kakao Local API가 추천 지역의 맛집 정보를 검색하며,
- ③ 다시 Gemini API가 두 결과를 종합하여 최종 여행 리포트를 Markdown으로 생성한다.

1-1. 개발 환경

항목	요구 내용	구현 위치 / 상태
개발 언어	Python 3.10 이상	
개발 도구	VSCode, Git	
LLM API	Google Gemini API (gemini-3.5-flash-lite)	llm_client.py
지도/장소 검색 API	Kakao Local API (키워드 기반 장소 검색)	map_client.py

2. 과제 목표 달성 체크리스트

미션 문서에서 요구한 4가지 학습 목표에 대해, 본 프로젝트를 통해 아래와 같이 설명 가능한 수준까지 학습 및 구현을 완료하였다.

항목	요구 내용	구현 위치 / 상태
REST API 구조 이해	요청/응답 구조와 GET/POST 메서드 차이 설명 가능	완료
구조화 출력 활용	LLM 출력(JSON)을 다음 단계(지도 검색) 입력으로 연결	완료
오류 대응 원칙	인증(401/403)/쿼터/네트워크/파싱 오류 대응 원칙 체득	완료
.env 키 관리	API 키를 코드에 직접 작성하지 않는 이유 및 방법 습득	완료

2-1. 학습 목표 상세 설명

① REST API의 요청/응답 구조와 GET/POST 차이

REST API는 클라이언트가 서버에 요청(Request)을 보내면 서버가 응답(Response)을 돌려주는 구조이다. 요청은 URL(엔드포인트), HTTP 메서드, 헤더, 파라미터로 구성되고, 응답은 상태 코드와 본문(JSON)으로 구성된다.

GET은 데이터 조회에 사용하며 파라미터가 URL에 노출되고 서버 상태를 변경하지 않는다(예: Kakao Local API 맛집 검색).

POST는 데이터 생성/제출에 사용하며 파라미터가 본문에 담겨 전달되고 새로운 결과를 생성한다(예:

Gemini API의 콘텐츠 생성 요청). 본 프로그램은 Kakao는 GET(검색), Gemini는 POST(생성) 방식으로 각각의 목적에 맞게 사용하였다.

② LLM 출력(JSON)을 다음 단계 입력으로 연결하는 흐름

본 프로그램의 핵심은 API 체이닝(Chaining) 구조이다.

1단계에서 Gemini가 여행 날짜를 입력받아 {"recommendations": [{"city": "제주", ...}]} 형태의 JSON을 생성하면, 이 중 city 값을 그대로 꺼내어 2단계 Kakao Local API의 검색어(입력 파라미터)로 사용한다.

이후 1·2단계 결과를 합쳐 3단계에서 Gemini에게 다시 전달하여 최종 리포트를 생성한다.

이러한 체이닝이 안정적으로 동작하려면 LLM 출력이 자유 텍스트가 아닌 일관된 JSON 스키마여야 하며, 이를 위해 response_mime_type="application/json" 옵션을 사용하였다.

③ 외부 API 호출의 대표 오류 유형과 대응 원칙

개발 과정에서 인증 오류(403, Kakao 카카오톡 서비스 미활성화), 파싱 오류(Gemini 응답이 유효한 JSON이 아닌 경우) 등을 실제로 겪었다.

대응 원칙은 다음 세 가지로 정리된다.

첫째, 한 단계가 실패해도 프로그램 전체를 중단시키지 않고 해당 부분만 "데이터 없음"으로 처리한 뒤 계속 진행한다(부분 실패 허용).

둘째, 모든 오류를 errors 리스트에 기록하여 추적 가능하게 한다.

셋째, LLM 응답 파싱 실패 시 최대 1회 재시도한다.

특히 모델 버전(gemini-2.5-flash → 404 오류)처럼 외부 요인으로 발생하는 오류는 코드 결함이 아니라 서비스 정책 변경일 수 있음을 실전에서 확인하였다.

④ API 키를 .env로 관리하는 이유

API 키를 코드에 직접 작성하지 않고 .env 파일과 python-dotenv로 분리 관리한 이유는 다음과 같다.

첫째, .env를 .gitignore에 등록함으로써 Git 커밋 이력에 키 값이 영구적으로 남는 것을 방지한다.

둘째, 로컬·배포 등 환경별로 다른 키를 코드 수정 없이 전환할 수 있다.

셋째, 값이 없는 형식만 담긴 .env.example을 공유함으로써 협업 시에도 실제 키 노출 없이 설정 방법을 안내할 수 있다.

실제로 check_api_keys() 함수가 키 미설정 시 프로그램을 즉시 종료시키도록 구현하여, 키가 코드에 하드코딩되어 있지 않음을 프로그램 동작으로 증명하였다.

3. 기능 요구사항 이행 현황

항목	요구 내용	구현 위치 / 상태
CLI 인터페이스	argparse, (표준라이브러리)필수 옵션 --date, 날짜 형식 검증	travel_planner.py / parse_args()
API 제공자 선택	Gemini + Kakao Local , 최소 응답 필드 확보	llm_client.py, map_client.py
.gitignore / README	불필요 파일 제외, 프로젝트 제목 작성	.gitignore, README.md

항목	요구 내용	구현 위치 / 상태
LLM 1차 추천 연동	JSON 스키마(city/weather/events/reason) 준수	get_initial_recommendation()
지도 API 맛집 검색	5곳 검색, 0건 시 중단 없이 다음 단계 진행	search_restaurants()
LLM 최종 리포트 생성	지역/이유/날씨/행사/맛집/일정 포함 Markdown	generate_final_report()
오류 처리(try-except)	키 미설정 종료, API 실패 시 데이터없음 처리, 재시도 1회	check_api_keys(), 각 클라이언트
API 키 관리(보안)	.env 사용(환경변수), 코드/제출물에 키 미노출.	.env, .gitignore
결과 저장	results/ 폴더에 원본 JSON + 리포트 MD 저장	cache_manager.save_cache()

4. 결과물 1 — CLI 기반 Python 프로그램

4-1. 프로젝트 구조

```

ravel_planner/
├── travel_planner.py    # 메인 CLI 프로그램 (진입점)
├── llm_client.py        # Gemini API 연동 (1차 추천 + 리포트 생성)
├── map_client.py        # Kakao Local API 연동 (맛집 검색)
├── cache_manager.py     # 결과 캐싱 (보너스 2)
├── .env                 # API 키 (Git 미포함)
├── .env.example         # 키 형식 예시 (Git 포함)
├── .gitignore           # .env, venv 등 제외
├── README.md           # 프로젝트 설명서
└── results/            # 실행 결과 저장 폴더

```

4-2. 실행 방법

```
python travel_planner.py --date "2026-03-15"
```

4-3. 실행 화면 (CLI 골격 - 날짜 검증 테스트)

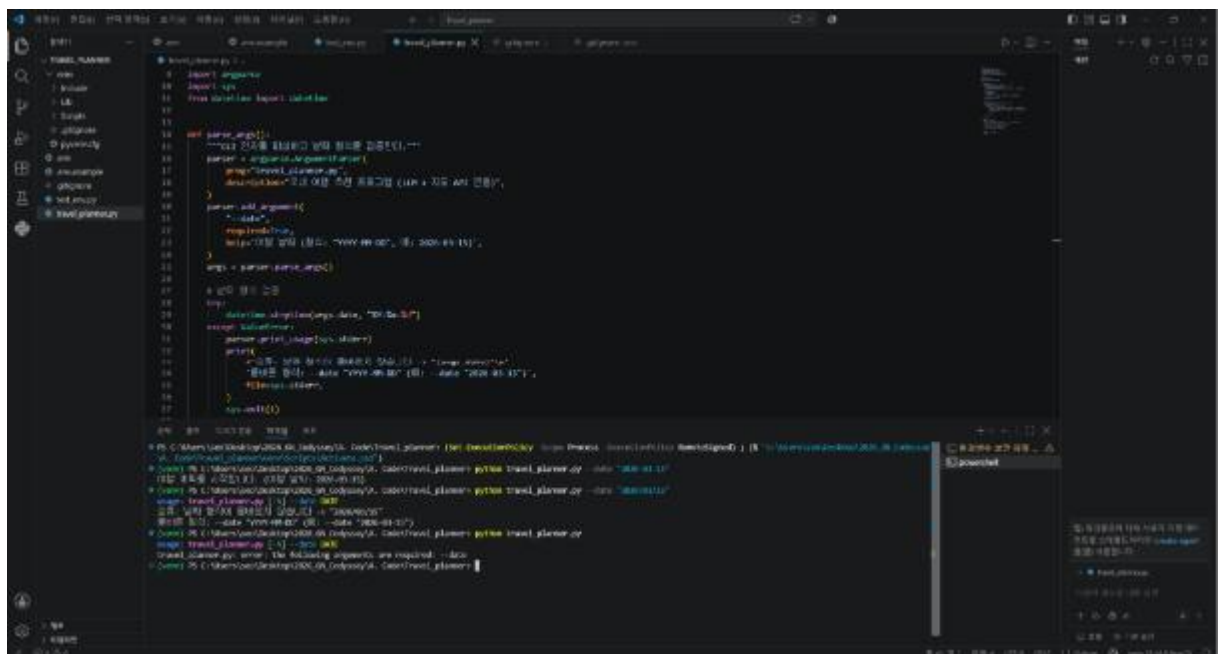


그림 1. argparse 기반 CLI 골격 및 날짜 형식 검증 테스트 결과 (STEP 4)

4-4. 핵심 로직 요약 — 날짜 검증

```
try:
    datetime.strptime(args.date, "%Y-%m-%d")
except ValueError:
    parser.print_usage(sys.stderr)
    print(f'오류: 날짜 형식이 올바르지 않습니다 -> "{args.date}"')
    sys.exit(1)
```

4-5. 핵심 로직 요약 — API 키 미설정 시 즉시 종료

```
def check_api_keys():
    missing = []
    if not os.getenv("GEMINI_API_KEY"): missing.append("GEMINI_API_KEY")
    if not os.getenv("KAKAO_REST_API_KEY"): missing.append("KAKAO_REST_API_KEY")
    if missing:
        print(" 다음 API 키가 설정되지 않았습니다:", ", ".join(missing))
        sys.exit(1)
```

5. 결과물 2 — 실행 결과 데이터 (results/)

실행 시 results/ 폴더에 실행 날짜를 기준으로 원본 데이터 JSON과 최종 리포트 Markdown이 자동 생성된다.

5-1. 원본 데이터 JSON 구조 (results/{날짜}_raw_data.json)

```
{
  "date": "2026-05-20",
  "recommendations": [
    {
      "city": "경주", "weather": "...", "events": [...],
      "reason": "...",
      "restaurants": [
        {
          "name": "...", "address": "...", "category": "...",
          "url": "...", "x": "...", "y": "...", ...
        }
      ]
    },
    { "city": "부산", ... }, { "city": "남해", ... }
  ],
  "errors": []
}
```

5-2. 최종 리포트 Markdown 구조 (results/{날짜}_travel_plan.md)

```
# 2026-05-20 국내 여행 추천 리포트
## 추천 지역
## 추천 이유
## 날씨 요약
## 행사/축제
## 맛집 추천
## (선택) 1일 일정 제안
## 오류 요약(errors)
```

5-3. 실행 화면 (LLM 1차 추천 + Kakao 맛집 검색 정상 동작)

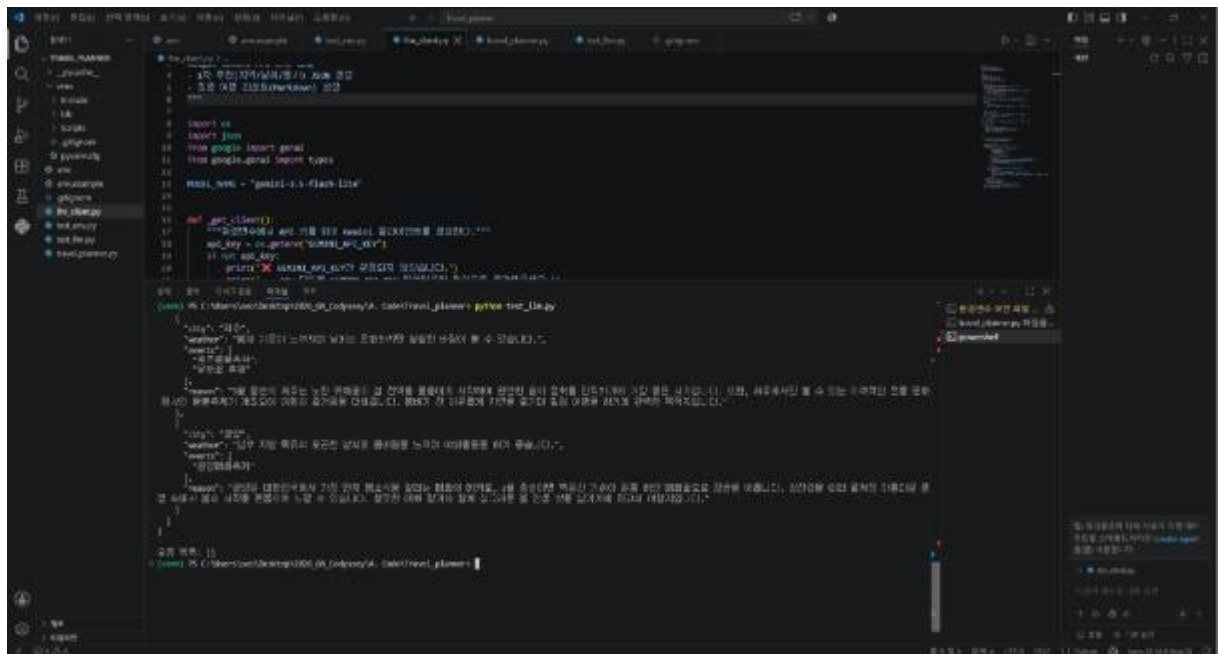


그림 2. Gemini API 1차 추천(제주/광양) 및 llm_client.py 정상 응답 확인 (STEP 5)

5-4. 전체 통합 실행 결과 로그 (STEP 11 통합 테스트)

```
$ python travel_planner.py --date "2026-05-20"
여행 계획을 시작합니다. (여행 날짜: 2026-05-20)
[1/3] 1차 추천 생성 중(LLM)...
- recommended_city: "경주"
- recommended_city: "부산"
- recommended_city: "남해"
[2/3] 맛집 검색 중(지도/장소 API)...
- '경주' 맛집 5곳 검색 완료
- '부산' 맛집 5곳 검색 완료
- '남해' 맛집 5곳 검색 완료
원본 데이터 저장 완료: results\2026-05-20_raw_data.json
[3/3] 최종 리포트 생성 중(LLM)...
- 리포트 생성 완료

완료! results\2026-05-20_travel_plan.md 를 확인하세요.
```

5-5. 보너스 2 — 캐싱 재실행 결과 (API 재호출 없음)

```
$ python travel_planner.py --date "2026-05-20"
여행 계획을 시작합니다. (여행 날짜: 2026-05-20)
[캐시 발견: results\2026-05-20_raw_data.json]
기존 원본 데이터를 재사용합니다. (Gemini/Kakao API 호출을 건너뛰니다)
[3/3] 최종 리포트 생성 중(LLM)...
- 리포트 생성 완료
```

6. 결과물 3 — README.md

README.md에는 아래 4가지 필수 항목을 모두 포함하였다.

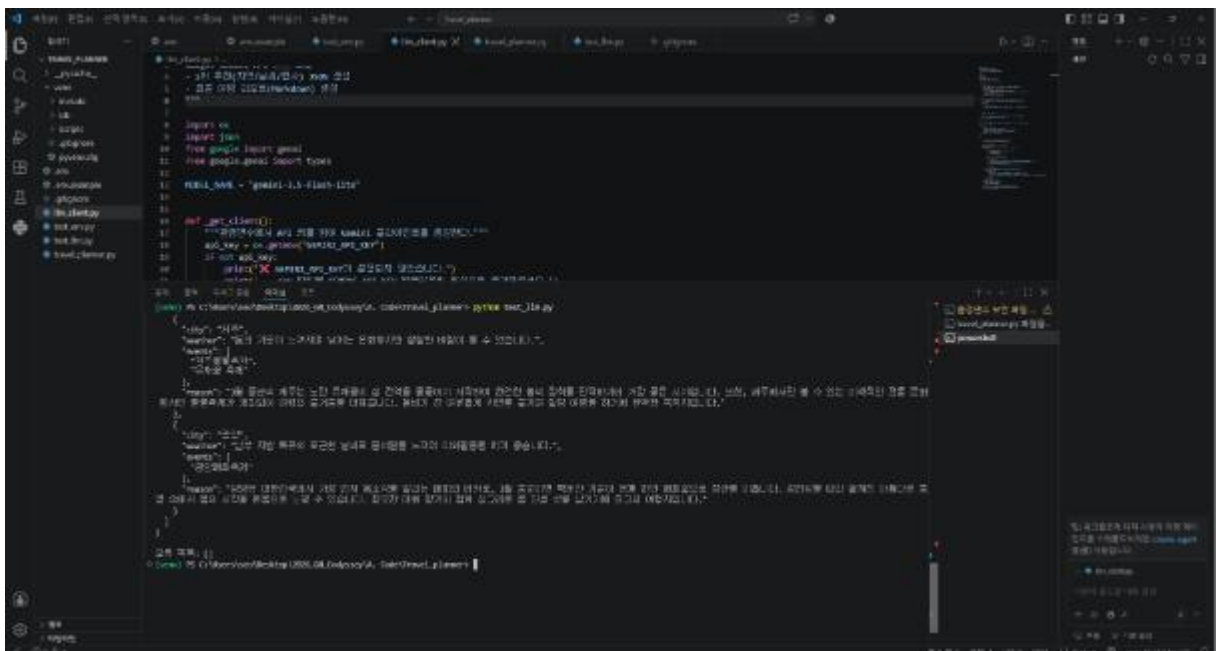
항목	요구 내용	구현 위치 / 상태
프로그램 개요	3단계 처리 흐름(1차 추천 → 맛집 검색 → 리포트 생성) 설명	README.md #1
실행 방법	가상환경 생성, 패키지 설치, 실행 커맨드 안내	README.md #3
API 키 설정 방법	Gemini/Kakao 키 발급처, .env 작성법 안내	README.md #4
결과물 확인 방법	results/ 폴더의 JSON/MD 파일 설명	README.md #5
보안 주의사항	키 미노출 원칙, .gitignore 등록, 키 재발급 안내	README.md #7

7. 보너스 과제 구현 내역 (2개 모두 진행)

7-1. 보너스 1 — 복수 지역 추천

1차 추천 JSON의 스키마를 recommendations 배열로 확장하여, Gemini가 한 번의 호출로 2~3개 지역을 동시에 추천하도록 프롬프트를 설계하였다. 이후 각 지역에 대해 반복문으로 맛집을 검색하고, 최종 리포트에서도 지역별 섹션으로 정리된다.

- 포인트: 반복 처리(for문) + API 요청 관리 + 다중 결과 구조 설계 경험
- 결과: 2026-05-20 실행 시 경주 / 부산 / 남해 3개 지역이 리포트에 함께 정리됨

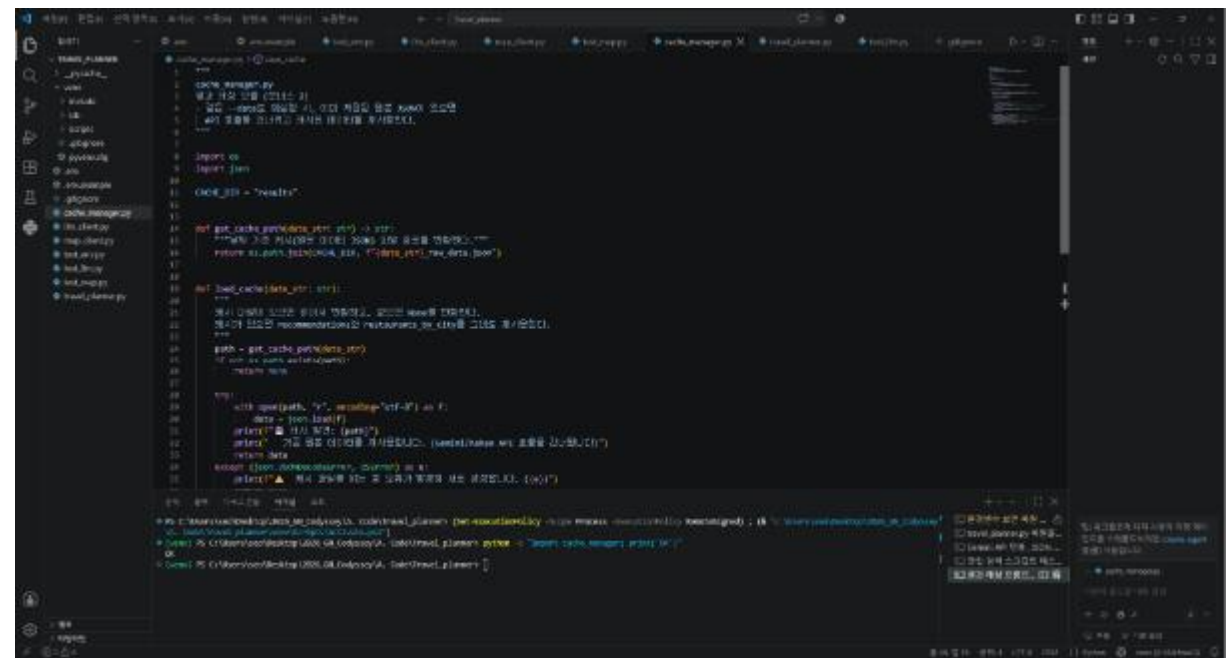


7-2. 보너스 2 — 결과 캐싱

cache_manager.py를 구현하여, 동일한 --date로 재실행 시 results/{날짜}_raw_data.json이 이미 존재하면 Gemini(1차 추천)와 Kakao(맛집 검색) API 호출을 모두 건너뛰고, 저장된 원본 데이터를 재사용

해 리포트만 재생성하도록 하였다.

- 포인트: 외부 API 비용/속도 최적화의 기본 원리 체득
- 결과: STEP 11 재실행 테스트에서 '🗺 캐시 발견' 메시지와 함께 API 재호출 없이 완료됨



8. 트러블슈팅 기록

개발 과정에서 발생한 주요 오류와 해결 과정을 기록한다. 이는 미션 목표 중 '외부 API 호출에서 발생하는 대표 오류와 대응 원칙 설명'과 직접 연결되는 실전 경험이다.

① Gemini 모델 404 NOT_FOUND

- 증상: gemini-2.5-flash 모델 호출 시 "This model ... available to new users" 404 오류 발생.
- 원인: Gemini 모델 세대가 3.x로 전환되며 구버전 모델이 신규 사용자에게 차단됨.
- 해결: 모델명을 gemini-3.5-flash-lite(2026년 8월 기준 GA, 저비용-저지연 모델)로 교체하여 해결.

② Kakao Local API 403 Forbidden

- 증상: 맛집 검색 요청 시 403 Forbidden, 응답 "disabled OPEN_MAP_AND_LOCAL service" 메시지.
- 원인: 카카오 개발자 콘솔에서 해당 앱에 '카카오맵(로컬)' 서비스 자체가 활성화되지 않은 상태였음.
- 해결: 콘솔에서 카카오맵 서비스 활성화 스위치 ON → 200 정상 응답 확인.

③ API 키 관리 오류 정책 구현

항목	요구 내용	구현 위치 / 상태
키 미설정	즉시 종료 + 설정 방법 안내 출력	check_api_keys()
지도 API 인증 실패	맛집=데이터 없음 처리 후 리포트 생성 진행	search_restaurants()
검색 결과 0건	중단 없이 데이터 없음으로 다음 단계 진행	search_restaurants()
LLM JSON 파싱 실패	재시도 1회, 실패 시 errors 배열에 기록	get_initial_recommendation()

9. 최종 결론 및 학습 성과

본 미션을 통해 단일 API 호출을 넘어, LLM API의 구조화된 출력을 다음 단계 API의 입력으로 연결하는 '체이닝(chaining)' 패턴을 직접 구현하였다. 이 과정에서 각 API의 인증 방식 차이(Gemini의 API 키 헤더 vs Kakao의 KakaoAK 헤더), 오류 유형별 대응 원칙(인증/쿼터/네트워크/파싱), 그리고 .env를 통한 키 관리의 실질적 필요성을 실제 트러블슈팅(403 Forbidden, 404 모델 만료)을 통해 체득하였다.

특히 Kakao Local API의 403 오류는 '키는 정상 발급되었지만 서비스가 비활성화된' 사례로, 인증 오류가 반드시 키 값 자체의 문제만은 아니라는 점을 실전에서 확인할 수 있었던 유의미한 경험이었다.

보너스 과제인 복수 지역 추천과 결과 캐싱을 모두 구현함으로써, 반복 처리 기반의 데이터 구조 설계와 외부 API 비용/속도 최적화라는 두 가지 배움 포인트를 추가로 확보하였다.

9-1. 최종 요구사항 이행 요약

항목	상태
CLI 기반 Python 프로그램 (argparse, 날짜 검증)	완료
결과 데이터 저장 (results/ JSON + Markdown)	완료
README.md (개요/실행법/키 설정/결과 확인/보안)	완료
LLM API 연동 - 1차 추천 (Gemini)	완료
지도/장소 검색 API 연동 - 맛집 검색 (Kakao Local)	완료
LLM API 연동 - 최종 리포트 생성 (Gemini)	완료
오류 처리(try-except, 재시도 1회)	완료
API 키 보안 관리 (.env, .gitignore)	완료
보너스 1 - 복수 지역 추천	완료
보너스 2 - 결과 캐싱	완료

※ 본 보고서에는 실제 실행 화면 캡처 3점(환경변수 로딩/CLI 검증, Gemini 1차 추천 응답)을 포함하였으며, 실행 로그는 개발 과정 중 터미널 출력을 그대로 인용하였다. API 키 값은 어디에도 노출되지 않았음을 확인하였다.

< 미션 수행 진행 과정 스크린샷 >

1단계. 프로젝트 폴더 생성, Python 가상환경(venv) 생성

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\user\Desktop\2026_08_Codysey\A. Code> cd "C:\Users\user\Desktop\2026_08_Codysey\A. Code"
PS C:\Users\user\Desktop\2026_08_Codysey\A. Code> mkdir Travel_planner

현재 위치: C:\Users\user\Desktop\2026_08_Codysey\A. Code

Mode                LastWriteTime         Length Name
----                -
d-----          2026-08-18 오후 9:14                Travel_planner

PS C:\Users\user\Desktop\2026_08_Codysey\A. Code> cd Travel_planner
PS C:\Users\user\Desktop\2026_08_Codysey\A. Code\Travel_planner> python --version
Python 3.10.4
PS C:\Users\user\Desktop\2026_08_Codysey\A. Code\Travel_planner> python -m venv venv
PS C:\Users\user\Desktop\2026_08_Codysey\A. Code\Travel_planner> .\venv\Scripts\activate.ps1
(PowerShell) PS C:\Users\user\Desktop\2026_08_Codysey\A. Code\Travel_planner> pip install google-genai requests python-dotenv
Collecting google-genai
  Using cached google_genai-2.17.0-py3-none-any.whl.metadata (56 kB)
Collecting requests
  Using cached requests-2.31.2-py3-none-any.whl.metadata (3.9 kB)
Collecting python-dotenv
  Using cached python_dotenv-1.2.0-py3-none-any.whl.metadata (27 kB)
Collecting anyio<4.0.0,>=4.0.0 (from google-genai)
  Using cached anyio-4.1.2-py3-none-any.whl.metadata (3.5 kB)
Collecting google-auth<3.0,>=2.25.0 (from google-genai)
  Using cached google_auth-2.36.1-py3-none-any.whl.metadata (6.0 kB)
Collecting httpx<3.0,>=2.0.1 (from google-genai)
  Using cached httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting pydantic<3.0,>=2.12.0 (from google-genai)
  Using cached pydantic-2.12.4-py3-none-any.whl.metadata (166 kB)
Collecting tenacity<9.2.0,>=8.2.3 (from google-genai)
  Using cached tenacity-8.2.3-py3-none-any.whl.metadata (1.3 kB)
Collecting annotated-types<1.0,>=1.0.0 (from google-genai)
  Using cached annotated_types-1.0.0-py3-none-any.whl.metadata (7.0 kB)
Collecting typing-extensions<4.0,>=3.10.0 (from google-genai)
  Using cached typing_extensions-4.12.2-py3-none-any.whl.metadata (3.3 kB)
Collecting charset-normalizer<4.0,>=3.0 (from requests)
  Using cached charset_normalizer-3.4.0-py3-none-any.whl.metadata (14 kB)
Collecting idna<4.0,>=3.0 (from requests)
  Using cached idna-3.10-py3-none-any.whl.metadata (9.7 kB)
Collecting urllib3<3.0,>=2.0.0 (from requests)
  Using cached urllib3-2.2.3-py3-none-any.whl.metadata (6.2 kB)
Installing collected packages: urllib3, typing-extensions, tenacity, annotated-types, python-dotenv, pydantic, anyio, requests, google-genai
Successfully installed annotated-types-0.7.0 anyio-4.12.2 certifi-2025.8.3 charset-normalizer-3.4.0 cryptography-45.0.0 distro-1.9.0 google-auth-2.36.1 httpx-0.28.1 idna-3.10 pydantic-2.12.4 pydantic-core-2.24.2 python-dotenv-1.2.0 requests-2.31.2 tenacity-8.2.3 typing-extensions-4.12.2 urllib3-2.2.3
(PowerShell) PS C:\Users\user\Desktop\2026_08_Codysey\A. Code\Travel_planner>
```

2단계. 필요 패키지 설치_ pip install google-genai requests python-dotenv

```
(PowerShell) PS C:\Users\user\Desktop\2026_08_Codysey\A. Code\Travel_planner> pip list
Package            Version
-----            -
annotated-types    0.7.0
anyio              4.12.2
certifi            2025.8.3
charset-normalizer 3.4.0
cryptography       45.0.0
distro             1.9.0
google-auth       2.36.1
google-genai      2.17.0
httpx              0.28.1
idna               3.10
pip                26.1.2
pydantic           2.12.4
pydantic_core     2.24.2
python-dotenv     1.2.0
requests          2.31.2
tenacity           8.2.3
typing-extensions 4.12.2
typing-inspection 0.0.2
urllib3           2.2.3
wheel             0.44.1

(PowerShell) PS C:\Users\user\Desktop\2026_08_Codysey\A. Code\Travel_planner> pip list
```

3단계. 공개 샘플 저장소 clone

```

arllib 2.7.8
setuptools 58.1.2
(venv) PS C:\Users\user\Desktop\2023_08_Codysey\A. Code\Travel_glaner> cd ..
(venv) PS C:\Users\user\Desktop\2023_08_Codysey\A. Code> git clone https://github.com/Trenkner/python-dataset.git sample_repo_check
Cloning into 'sample_repo_check'...
remote: Enumerating objects: 262, done.
remote: Counting objects: 1886 (960/926), done.
remote: Compressing objects: 1886 (1115/771), done.
remote: Total 2628 (delta 498), reused 0 (delta 498), pack-reused 158 (from 3)
Receiving objects: 2628 (2480/148), 1.20 MiB/s | 2.81 MiB/s, done.
Resolving deltas: 1886 (1136/750), done.
(venv) PS C:\Users\user\Desktop\2023_08_Codysey\A. Code> cd sample_repo_check
(venv) PS C:\Users\user\Desktop\2023_08_Codysey\A. Code\sample_repo_check> git log --oneline -10
791f0c2 (HEAD -> main, origin/main, origin/HEAD) chore(actions/checkout): bump actions/checkout from 4.0.1 to 4.0.2 in the github-actions group (#604)
7193716 chore(deps): update actions-include-macos-plugin requirement from >= 0.0 to >= 0.0 (#603)
8560702 chore(deps): update pytest requirement from >= 0.0 to >= 0.0 (#602)
7206a41 ci(changes): bump python/dataset-gi-pages from 0.0.0 to 0.0.0 in the github-actions group (#601)
8781226 ci(changes): bump ci/cd setup with 509 pinning and least-privilege permission (#598)
3f6e1e8 Cache empty dataset dict (#600)
8c4e0d1 fix: setup gpr-0.0.0 from .env files (#596)
806a2d0 fix: type to comment (#621)
74b0e08 Add missing interpolate parameter to the decorator of function 'load_dataset' and 'dataset_values' (#601)
(venv) PS C:\Users\user\Desktop\2023_08_Codysey\A. Code\sample_repo_check> dir

Dir 지크 : C:\Users\user\Desktop\2023_08_Codysey\A. Code\sample_repo_check

Mode                LastWriteTime         Length Name
----                -
d-----          2023-08-10 오전 9:22             .github
d-----          2023-08-10 오전 9:22             docs
d-----          2023-08-10 오전 9:22             src
d-----          2023-08-10 오전 9:22             tests
->          2023-08-10 오전 9:22             121 .isort.cfg
->          2023-08-10 오전 9:22             268 .isort.cfg
->          2023-08-10 오전 9:22             1696 .gitignore
->          2023-08-10 오전 9:22             190 .pre-commit-config.yaml
->          2023-08-10 오전 9:22             1928 CHANGES.md
->          2023-08-10 오전 9:22             795 CODEOWNERS.md
->          2023-08-10 오전 9:22             1881 LICENSE
->          2023-08-10 오전 9:22             631 README.md
->          2023-08-10 오전 9:22             396 README.md
->          2023-08-10 오전 9:22             868 dataset.py
->          2023-08-10 오전 9:22             2229 pyproject.toml
->          2023-08-10 오전 9:22             8123 README.md
->          2023-08-10 오전 9:22             135 requirements-test.txt
->          2023-08-10 오전 9:22             61 requirements.txt
->          2023-08-10 오전 9:22             197 setup.py
->          2023-08-10 오전 9:22             1271 test.py

```

4단계. 환경변수 파일로 분리하는 보안 설정 단계(.env / .gitignore/.env.example/test_env.py)

```

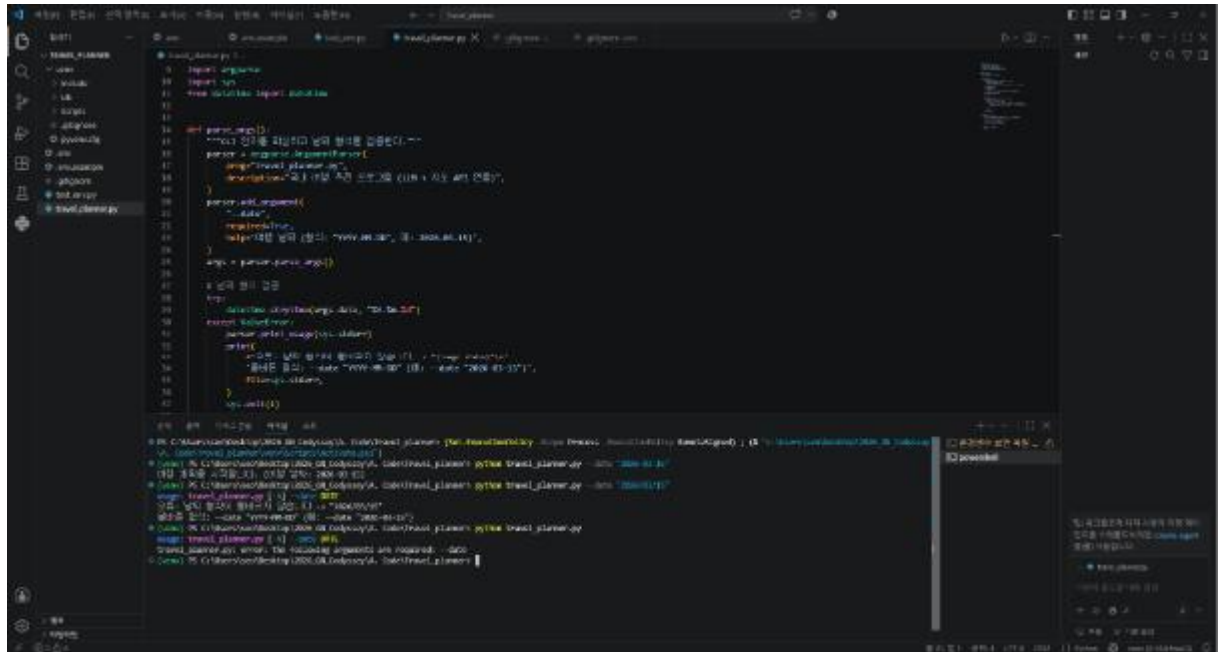
(venv) PS C:\Users\user\Desktop\2023_08_Codysey\A. Code\Travel_glaner> cd ..
(venv) PS C:\Users\user\Desktop\2023_08_Codysey\A. Code> git clone https://github.com/Trenkner/python-dataset.git sample_repo_check
Cloning into 'sample_repo_check'...
remote: Enumerating objects: 262, done.
remote: Counting objects: 1886 (960/926), done.
remote: Compressing objects: 1886 (1115/771), done.
remote: Total 2628 (delta 498), reused 0 (delta 498), pack-reused 158 (from 3)
Receiving objects: 2628 (2480/148), 1.20 MiB/s | 2.81 MiB/s, done.
Resolving deltas: 1886 (1136/750), done.
(venv) PS C:\Users\user\Desktop\2023_08_Codysey\A. Code> cd sample_repo_check
(venv) PS C:\Users\user\Desktop\2023_08_Codysey\A. Code\sample_repo_check> git log --oneline -10
791f0c2 (HEAD -> main, origin/main, origin/HEAD) chore(actions/checkout): bump actions/checkout from 4.0.1 to 4.0.2 in the github-actions group (#604)
7193716 chore(deps): update actions-include-macos-plugin requirement from >= 0.0 to >= 0.0 (#603)
8560702 chore(deps): update pytest requirement from >= 0.0 to >= 0.0 (#602)
7206a41 ci(changes): bump python/dataset-gi-pages from 0.0.0 to 0.0.0 in the github-actions group (#601)
8781226 ci(changes): bump ci/cd setup with 509 pinning and least-privilege permission (#598)
3f6e1e8 Cache empty dataset dict (#600)
8c4e0d1 fix: setup gpr-0.0.0 from .env files (#596)
806a2d0 fix: type to comment (#621)
74b0e08 Add missing interpolate parameter to the decorator of function 'load_dataset' and 'dataset_values' (#601)
(venv) PS C:\Users\user\Desktop\2023_08_Codysey\A. Code\sample_repo_check> dir

Dir 지크 : C:\Users\user\Desktop\2023_08_Codysey\A. Code\sample_repo_check

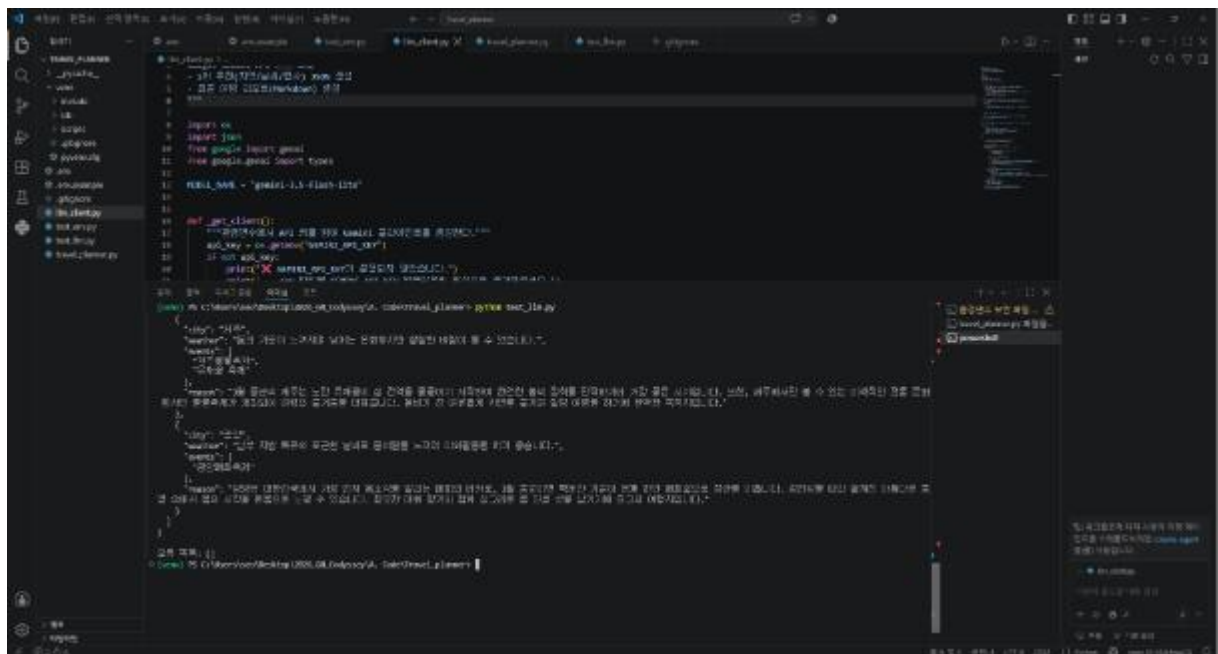
Mode                LastWriteTime         Length Name
----                -
d-----          2023-08-10 오전 9:22             .github
d-----          2023-08-10 오전 9:22             docs
d-----          2023-08-10 오전 9:22             src
d-----          2023-08-10 오전 9:22             tests
->          2023-08-10 오전 9:22             121 .isort.cfg
->          2023-08-10 오전 9:22             268 .isort.cfg
->          2023-08-10 오전 9:22             1696 .gitignore
->          2023-08-10 오전 9:22             190 .pre-commit-config.yaml
->          2023-08-10 오전 9:22             1928 CHANGES.md
->          2023-08-10 오전 9:22             795 CODEOWNERS.md
->          2023-08-10 오전 9:22             1881 LICENSE
->          2023-08-10 오전 9:22             631 README.md
->          2023-08-10 오전 9:22             396 README.md
->          2023-08-10 오전 9:22             868 dataset.py
->          2023-08-10 오전 9:22             2229 pyproject.toml
->          2023-08-10 오전 9:22             8123 README.md
->          2023-08-10 오전 9:22             135 requirements-test.txt
->          2023-08-10 오전 9:22             61 requirements.txt
->          2023-08-10 오전 9:22             197 setup.py
->          2023-08-10 오전 9:22             1271 test.py

```

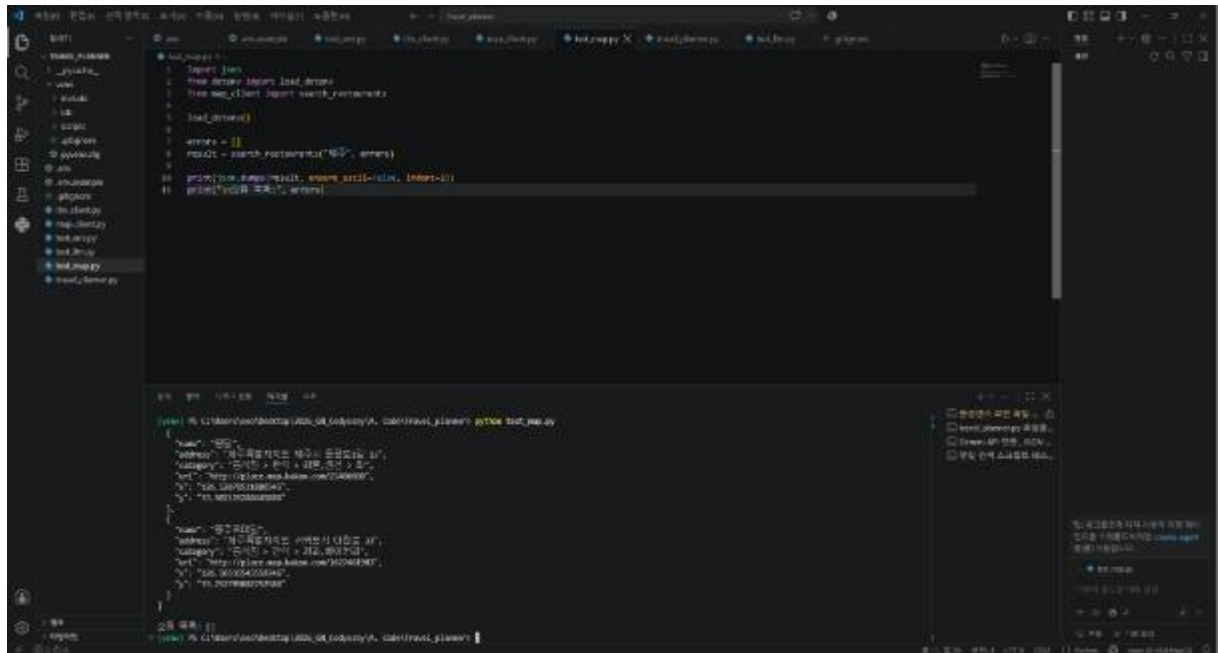
5단계. travel_planner.py 파일을 생성, 실행 테스트



6단계. Gemini API 연동_ JSON 출력_ 실행 테스트



7단계. 맛집 검색 스크립트 작성 실행 테스트



```
def search_restaurants(location, radius):
    """
    Search for restaurants near a given location within a specified radius.
    """
    # Define the REST API endpoint
    url = f"http://api.yelp.com/v3/search?location={location}&radius={radius}"

    # Define the headers for the REST API call
    headers = {
        "Authorization": "Bearer YOUR_API_KEY_HERE",
        "Content-Type": "application/json"
    }

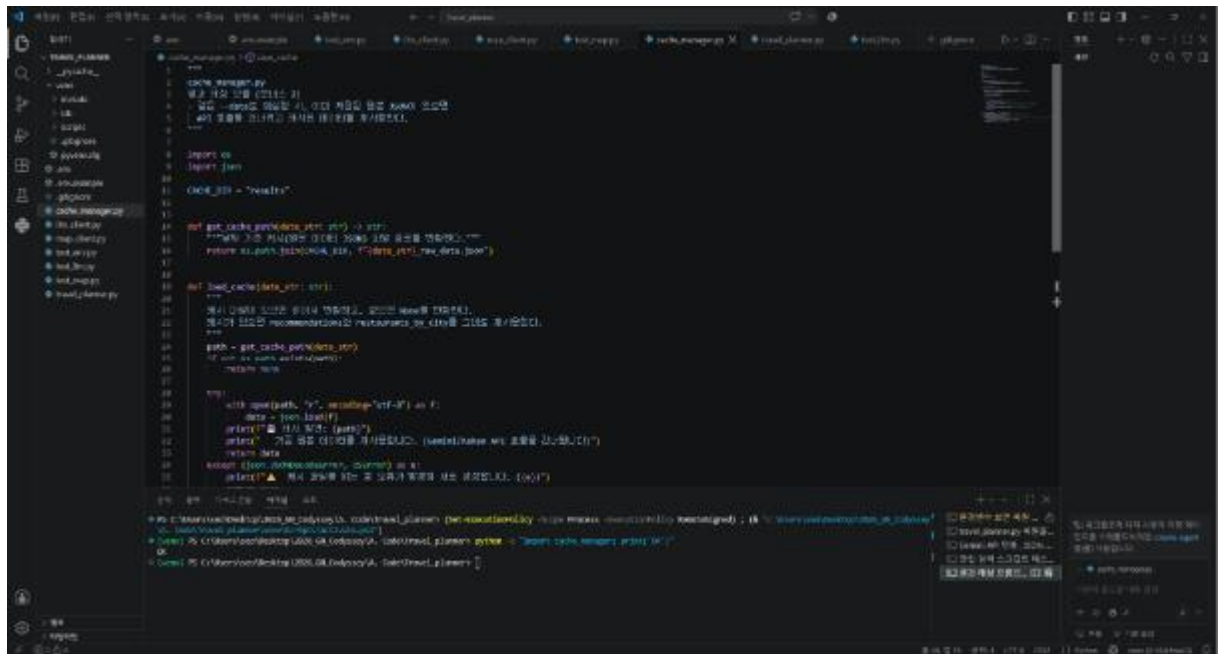
    # Make the REST API call
    response = requests.get(url, headers=headers)

    # Check if the request was successful
    if response.status_code == 200:
        # Parse the JSON response
        data = response.json()

        # Extract the search results
        results = data.get("businesses", [])

        # Print the search results
        print(f"Search results for {location} within {radius} km:")
        for result in results:
            print(result)
    else:
        # Handle the error
        print(f"Error: {response.status_code} {response.text}")
```

8단계. 결과 캐싱 _이미 저장된 1차 추천 JSON이 있으면 API 호출(Gemini)을 건너뛰고 그 데이터를 재사용



```
def search_restaurants(location, radius):
    """
    Search for restaurants near a given location within a specified radius.
    """
    # Define the REST API endpoint
    url = f"http://api.yelp.com/v3/search?location={location}&radius={radius}"

    # Define the headers for the REST API call
    headers = {
        "Authorization": "Bearer YOUR_API_KEY_HERE",
        "Content-Type": "application/json"
    }

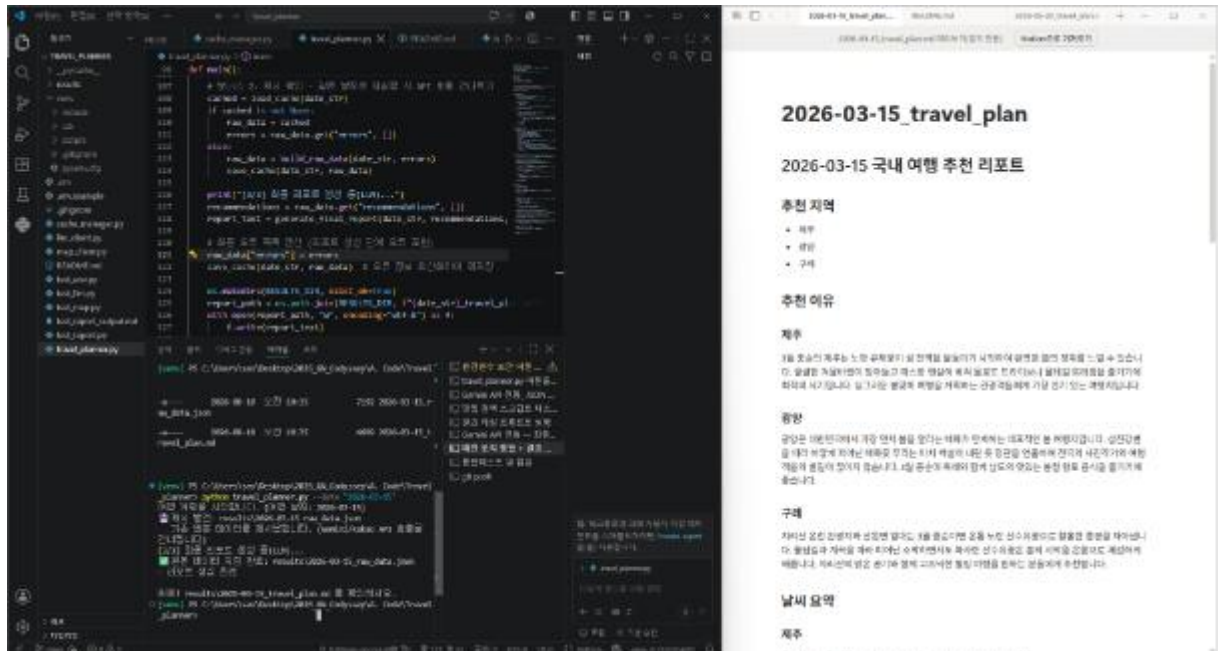
    # Make the REST API call
    response = requests.get(url, headers=headers)

    # Check if the request was successful
    if response.status_code == 200:
        # Parse the JSON response
        data = response.json()

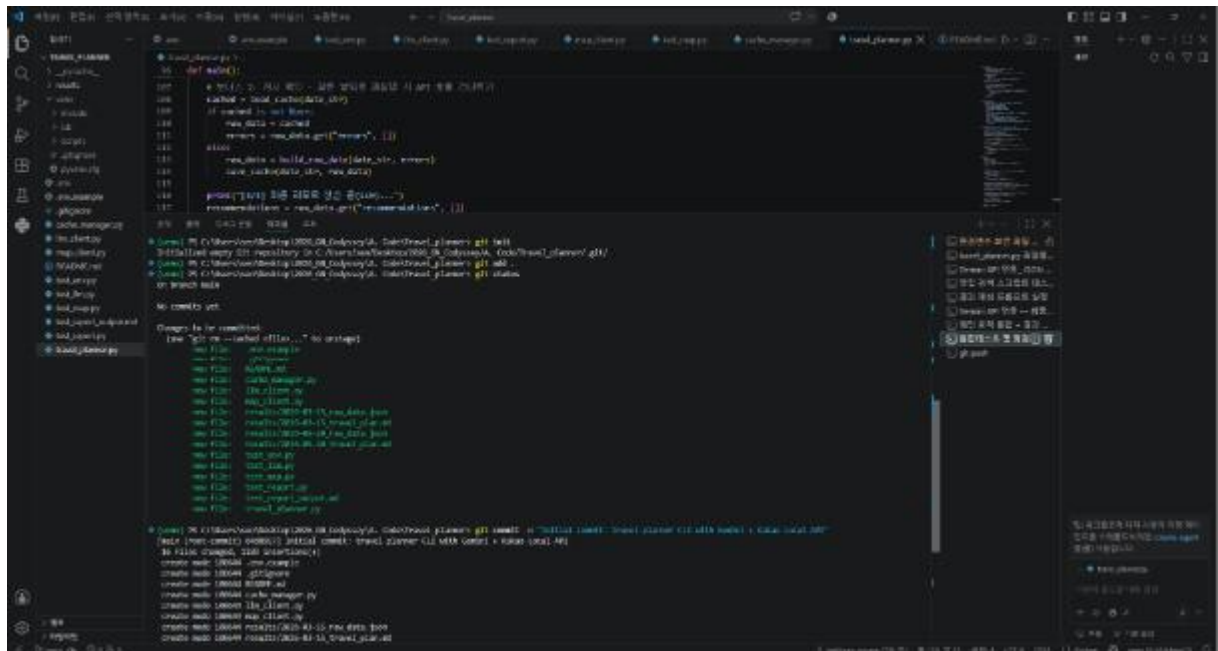
        # Extract the search results
        results = data.get("businesses", [])

        # Print the search results
        print(f"Search results for {location} within {radius} km:")
        for result in results:
            print(result)
    else:
        # Handle the error
        print(f"Error: {response.status_code} {response.text}")
```

9단계. Gemini API 연동 — 최종 리포트 생성 (Markdown)



10단계. Git 보안 점검 (매우 중요)



11단계. Git push

- 1) git init
- 2) git add .
- 3) git status
- 4) git push https://github.com/bzkhseo-source/Python-Application_-Domestic-Travel-Recommendation-Program.git main

